41052 Advanced Algorithms

Makespan Scheduling on Identical Machines

Dynamic Programming

vs

Longest Processing Time

https://youtu.be/PVLWi72qLqE?si=MNaWgvmeJBTN4Xst

Tutor's name: Dr Troy Lee

Thomas Page - 26194035

Anna Lazarevic - 24963326

1. Problem and Algorithms **3**
1.1 Problem Definition 3
1.2 Exact Dynamic Programming - Thomas 3
Implementation 3
Complexity 4
Exactness 4
1.3 LPT Approximation - Anna 4
Implementation 5
Complexity 5
Guarantee 5
2. Theoretical Comparison and Predictions **6**
2.1 Theoretical Comparison 6
2.1.1 Constant factors and memory 6
2.1.2 Input features and sensitivity 7
2.2 Predictions Before Experiments 8
3. Experimental Methodology **9**
3.1 Environment 9
3.2 Input Families 9
3.3 Experiments 10
3.4 Measurements 10
4. Results and Analysis **12**
4.1 Runtime 12
4.2 Solution Quality 14
4.3 Effect of Input Structure 16
4.4 Which Algorithm Wins, and Where 18
5. What Surprised Us **20**
5.1 DP 20
5.2 LPT 20
6. What We Would Do Next **22**
7. Individual Contributions **22**
Thomas 22
Anna 22
Shared 22
8. AI Use **23**
8.1 Thomas 23
8.2 Anna 23
8.3 AI in Group Work 24
8.3.1 Where AI was wrong - Benchmarking Refactor 24
8.3.2 Where AI was wrong - Misleading Optimisation Advice 24
8.3.3 Where AI was wrong - Gemini Failing to Identify Errors in AI-Generated Code 25
8.4 Reflection of AI in group work 25

1. Problem and Algorithms

1.1 Problem Definition

Makespan scheduling on identical machines considers a set of n independent jobs to be assigned across m identical machines. Each job j has a processing time p_j, representing the amount of time required for that job to complete. Each job must be assigned to exactly one machine, while each machine may process multiple jobs. Since the machines operate in parallel, the completion time of a machine is determined by the total processing time of the jobs assigned to it.

Let L_i represent the total load assigned to machine i. The makespan, denoted by C_{max}, is the completion time of the most heavily loaded machine:

$$C_{max} = \max_i L_i$$

The objective is to assign the jobs so that C_{max} is minimised, producing the most balanced schedule possible.

The problem becomes computationally difficult because the number of possible job assignments grows rapidly with the number of jobs and machines. With n jobs and m machines, there can be up to m^n possible assignments to consider in a direct exhaustive search. Consequently, finding a guaranteed optimal schedule becomes impractical for sufficiently large instances, motivating the comparison between an exact dynamic programming approach and the faster LPT approximation algorithm.

1.2 Exact Dynamic Programming - Thomas

The exact algorithm uses a top-down dynamic programming (DP) approach using memoisation over machine-load states. At any point, the state is represented as dp(job_index, loads), where job_index identifies the next job to assign and loads records the current load of each machine. The algorithm recursively assigns the next job to each distinct machine load and returns the smallest final makespan achievable from that state.

Since each machine is identical, the load tuple is sorted before being used as a cache key. This means that equivalent states such as (2,5) and (5,2) are treated as the same subproblem. The algorithm is therefore exact, but avoids recomputing schedules that differ only by machine ordering.

Implementation

An lru_cache is used to memoise previously solved states. For each job, the algorithm creates a new load configuration for every distinct machine choice, the tuple is sorted, and recursively evaluates the remaining jobs.

Two additional reductions are used. First, if several machines currently have the same load, only one of those equivalent assignments is explored. Second, the algorithm uses the lower bound.

$$LB = max(max_i p_i, \lceil (\sum_i p_i)/m \rceil)$$

And stops searching a state once a completion reaches this value, since no schedule can achieve a smaller makespan.

After the optimal value has been found, the schedule is reconstructed by following transitions whose cached results are consistent with the optimum. Actual machine identities are maintained separately from the sorted DP states so that a concrete job assignment can still be returned.

Complexity

Let R denote the number of distinct memoised states and m be the number of machines. Each state considers at most m assignments, and each transition constructs and sorts an m-element load tuple. This gives an upper bound of:

$$O(R \cdot m^2 \log m) \text{ time}$$

And

$$O(Rm) \text{ space}$$

For non-negative integer processing times with total processing time P, a loose upper bound on the number of states is:

$$R = O((n + 1)(P + 1)^{(m-1)})$$

For fixed m, this makes the algorithm pseudo-polynomial in P. In general, however, the state space can still grow exponentially, which limits the algorithm to relatively small instances compared with LPT.

Exactness

The algorithm considers every distinct placement of each job and selects the best resulting completion, so the returned makespan is optimal. Memoisation only removes repeated computation, while sorting and equal-load skipping remove schedules that are equivalent under permutation of identical machines. Lower-bound stopping is also safe because it only terminates a search once the theoretical minimum possible makespan has already been reached.

1.3 LPT Approximation - Anna

Longest Processing Time First (LPT) is a greedy approximation algorithm that sorts jobs in descending order of processing time and assigns each job to the machine with the smallest current load. The intuition is that longer jobs have greater influence on the final makespan, so assigning them first provides more opportunity to balance the workload across machines. Shorter jobs are then placed into the remaining capacity, helping to even out the final loads.

The sorting addresses a key weakness of list scheduling, which applies the same assignment rule without sorting the jobs first. If several short jobs are assigned before a long job arrives, the long job may be placed on an already heavily loaded machine, resulting in a makespan that approaches twice the optimum. By

prioritising longer jobs, LPT reduces the likelihood of this imbalance and provides a stronger approximation guarantee.

Implementation

The implementation in lpt.py keeps the machines in a min-heap of (load, machine index) pairs. For each job, the algorithm removes the least-loaded machine, assigns the job to it, and reinserts the machine with its updated load. Python's lexicographic tuple comparison ensures that ties in machine load are resolved by selecting the machine with the lowest index, making the assignment deterministic.

Complexity

Sorting n jobs takes O(n log n) time, while assigning each job requires O(log m) time for the heap operations. The overall time complexity is therefore O(n log n + n log m), which simplifies to O(n log n) when $m \leq n$. The space complexity is O(n + m), accounting for the sorted jobs, machine heap and resulting schedule. The running time depends on the number of jobs and machines rather than the magnitude of their processing times.

Guarantee

LPT guarantees a makespan at most $(4/3 - 1/(3m)) \times \text{OPT}$, where m is the number of machines and OPT is the optimal makespan. This bound is tight, meaning that there are instances for which LPT achieves this worst-case ratio.

2. Theoretical Comparison and Predictions

The two algorithms take fundamentally different approaches to the same problem. DP explores the space of possible assignments, reusing results for repeated load states to guarantee an optimal schedule; however, this guarantee comes at the cost of a running time that can grow exponentially. In contrast, LPT makes a single pass through the sorted jobs, committing to each assignment without reconsidering previous decisions. This keeps its running time low, but the resulting schedule may be suboptimal. Neither algorithm is universally superior as DP prioritises solution quality, while LPT prioritises computational efficiency.

2.1 Theoretical Comparison

Feature	Exact DP	LPT
Solution quality	Optimal	Approximate
Runtime	Exponential / state-dependent	Polynomial
Memory	Potentially large	Small
Main strength	Guaranteed optimum	Speed and scalability
Main weakness	Scalability	May be suboptimal

2.1.1 Constant factors and memory

The DP memoises states of the form (job index, load tuple), reusing results when different partial assignments produce the same machine loads. Without memoisation, it would explore all m^n possible assignments. Because the machines are identical, the implementation sorts each load tuple before caching, so states that differ only in the order of their loads, such as (5, 3) and (3, 5), are treated as one. It also skips machines whose load equals the previous machine's, since assigning the job to either produces the same state. The number of distinct states still depends on the job times: small times produce many repeated loads and therefore many reusable states, while large, varied times reduce this overlap and push the search towards its exponential worst case.

The implementation also stops early when it finds a schedule whose makespan equals a simple lower bound on the optimum: the larger of the longest job and the average load, rounded up. When a perfectly balanced schedule exists, this can end the search well before every state is explored. When the optimum is above this bound, however, no early stop occurs and every reachable state must still be visited.

Each state carries significant overhead: a cache lookup and key hash, plus, for each of up to m machines, copying the load list, sorting it and building a new tuple. This adds $O(m^2 \log m)$ work per state on top of the function call itself. Since the cache retains every visited state, memory use grows alongside running time.

LPT's two main operations, sorting and heap management, use Python's built-in sorted and heapq, which are implemented in C and run much faster than equivalent Python code. This means each job requires only a small number of efficient operations. Its memory use depends only on the input size: the job list, a heap of m entries and the resulting schedule.

The difference between the algorithms therefore extends beyond their asymptotic complexities to their constant factors. Even when the DP explores relatively few states, processing each state involves considerably more work than a single LPT step.

2.1.2 Input features and sensitivity

Input feature	Effect on DP*	Effect on LPT
Number of jobs n	**Runtime**: grows exponentially when job times are large and varied; more slowly when they are small.	**Runtime**: grows as O(n log n), mostly from the initial sort. **Quality**: more jobs means more small jobs to fill gaps, so results move closer to optimal.
Number of machines m	**Runtime**: grows exponentially, since each job has m choices and the number of load states rises with m.	**Runtime**: grows only as log m per job, through the heap. **Quality**: fewer jobs per machine leaves less room to balance loads.
Job magnitude	**Runtime**: larger times create more distinct load states, so fewer are reused and more must be explored.	**Runtime**: no effect; large numbers are compared as quickly as small ones. **Quality**: no effect; scaling every job by the same amount gives the same schedule, scaled.
Job size spread	**Runtime**: affects runtime only through the total of the job times.	**Runtime**: no effect. **Quality**: a few large jobs with many small ones suits LPT; similar-sized jobs leave less room to correct early imbalances.
Jobs per machine (n/m)	**Runtime**: no effect beyond n and m themselves.	**Runtime**: no effect beyond n and m themselves. **Quality**: the main driver of LPT's error; fewer jobs per machine means larger errors.

**The DP's solution quality is always optimal, so its column describes runtime only.*

Overall, the DP's cost is driven by the values of the jobs, while LPT's quality is driven by how many jobs share each machine, so no single input size determines which algorithm is preferable.

2.2 Predictions Before Experiments

DP:

- DP will find the optimal makespan for instances it can finish.
- More jobs will generally increase runtime and memory use.
- More machines will generally increase the number of load states.
- Repeated job times should produce fewer distinct states than varied times.
- Early stopping should help when the optimum matches the lower bound.
- DP should use more time and memory than LPT, with equal or better makespans.
- DP should be practical mainly for smaller instances.
- Inputs with the same job and machine counts may have very different runtimes.
- Memoisation should reduce repeated computation compared with uncached recursion.
- Sorting load tuples should reduce cached states compared with keeping machine order.
- Skipping equal-load machines should reduce redundant cache lookups.
- Early stopping should offer no benefit when the optimum exceeds the lower bound.
- Changing job order may affect runtime but should preserve the optimal makespan.
- Multiplying all job times by the same positive integer should preserve the state count.

LPT:

- The LPT will run under 1ms when n <= 1000 and under 0.5 seconds for n = 100,000 staying fast on instances too large for DP.
- For a fixed n and m, the runtime won't change as job times increase.
- On random inputs the LPT will match the optimum on most instances and have an average makespan no more than 5% above the optimum.
- The LPT's error will be largest at around 2-3 jobs per machine.
- The LPT's error will be almost always 0 at 5 or more jobs per machine.
- LPT will be closer to the optimal makespan on inputs with a few large jobs and many small ones compared to those where all jobs are similar size.
- LPT will reach exactly (4/3 -1/(3m)) * OPT on the worst case family.

Crossover

- LPT will be faster than the DP on every instance tested.
- The DP will finish within 10 s for much larger n with two machines than with three or more.

3. Experimental Methodology

3.1 Environment

All experiments were run on a single machine so runtimes are directly comparable.

- Hardware: AMD Ryzen AI 7 350 (8 cores, 16 threads, 2.0 GHz base clock), 32 GB RAM
- Operating system: Windows 11, 64-bit
- Python version: 3.14.3
- Libraries: pytest, matplotlib, standard library modules (heapq, functools, time, tracemalloc and multiprocessing)
- Timing method: time.perf_counter(), measured around the solve call only, so input generation and file output are excluded
- Repetitions / warm-up: Each timing is the median of several repeated runs (5–11 for LPT, up to 3 for the DP; DP runs over 1s were timed once, since their variation is small relative to their length). The run order was shuffled with a fixed seed, as an early version of E8 showed that runs at the start of a session were systematically slower.
- Timeout: each DP run was executed in a separate process with a 10-second limit; runs exceeding it were recorded as "did not finish" (DNF) rather than discarded.
- Memory: peak memory was measured with tracemalloc in separate runs, since memory tracking slows execution and would distort the timings.

The DP's cache is created inside each call, so every run starts with an empty cache and no results carry over between instances.

3.2 Input Families

All instances were produced by seeded generators in benchmarks/generators.py, so every instance can be regenerated exactly.

Family	How its Generated	Why its included
Uniform random	N job times drawn uniformly from 1 to T	Neutral baseline for runtime and solution quality.
Repeated sizes	Job times drawn from only 3 distinct values	Tests whether repeated sizes increase DP state reuse
Highly varied	All job times distinct, from a wide range	Minimises DP state reuse
A few large + many small	A small number of jobs from a high range, the rest from a low range	Tests whether small jobs help balance LPT loads

Similar sized	Job times from a narrow range (45-55)	Leaves LPT little room to correct imbalances
Balanced	Target load is split into random pieces for each machine, then all pieces are shuffled, so a perfect split is guaranteed.	Tests the DP's early stopping
LPT worst	Two jobs each of sizes 2m - 1, 2m - 2, ..., m + 1, plus three jobs of size m. The optimal makespan is 3m; LPT gives 4m - 1	Shows LPT at its theoretical worst

3.3 Experiments

ID	Experiment	What varies
E1	Scaling	N and m
E2	Job magnitude	Maximum job time, and multiplying all job times by a constant
E3	LPT quality	Jobs per machine (n/m)
E4	Input structure	Input family, at fixed n and m
E5	Early stopping	Balanced vs unbalanced instances early stopping on and off
E6	DP design choices	DP variants; no cache, cache, sorted loads, skipping equal loads, early stopping
E7	Job order	Original, ascending, descending and shuffled job order
E8	LPT at scale	n up to 100,000 (LPT only)
E9	LPT worst case	m from 2 - 10
E10	DP feasibility by input structure	Job count, machine count, and repeated/uniform/varied inputs under a fixed 10-second DP limit

3.4 Measurements

For each instance, we recorded:

- The makespan from each algorithm

- The runtime of each algorithm
- The DP's number of cached states and number of calls
- Peak memory for each algorithm
- LPT relative error compared with the optimum.

$$\textbf{Relative error} = (C_{LPT} - C_{OPT}) / C_{OPT} \times 100\%$$

Fairness rule: both algorithms must receive the same generated instances, and timing should exclude unrelated I/O where possible.

4. Results and Analysis

4.1 Runtime

Figure 1 shows that the number of machines had a greater effect on the DP than the number of jobs. With two machines, the DP stayed below 1ms across the whole range of up to 40 jobs. With three machines, it rose to a few tens of milliseconds around n = 28 and then fell again. With four machines, it reached several seconds, and from n = 16 onwards, some of the runs hit the 10-second limit.

The DP's runtime did not increase steadily with n for three or four machines. This follows from the DP's early stopping, with more jobs, a schedule matching the lower bound becomes more likely to exist and quicker to find, so the search can end sooner even though there are more jobs to place.

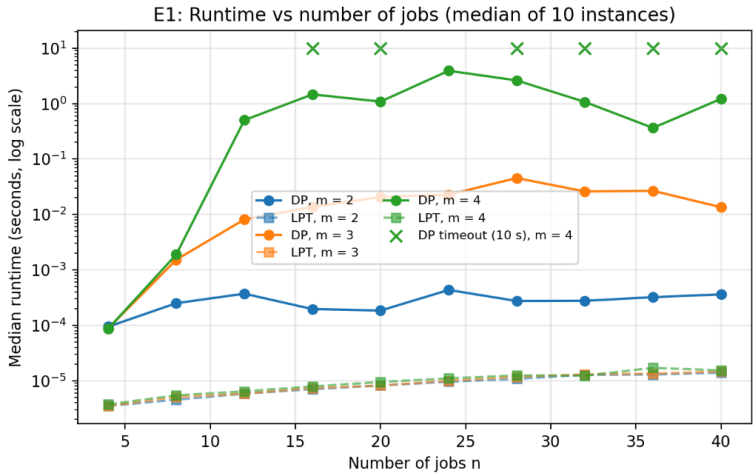


Figure 1: Median runtime of the DP and LPT against the number of jobs, for 2, 3 and 4 machines (E1). Crosses mark DP runs stopped at the 10-second limit.

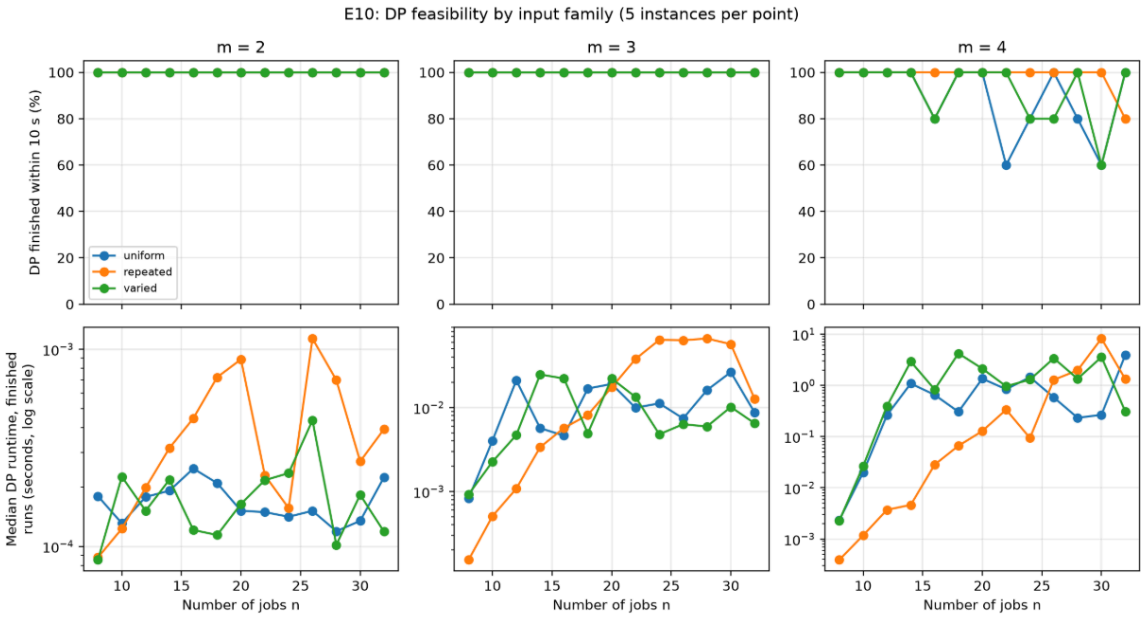


Figure 2: Percentage of instances the DP finished within 10 seconds (top) and its median runtime on those it finished (bottom), by input family, number of jobs, and number of machines.

Figure 2 shows that machine count has the strongest effect on DP feasibility. With two or three machines, every tested instance completed across all input families up to n = 32. With four machines, timeouts first appeared around n = 16 for uniform and varied inputs, and completion became increasingly dependent on the particular instance. Repeated-size inputs remained more reliable, completing all instances through n = 30 and 80% at n = 32, demonstrating how repeated processing times reduce the number of distinct states and extend the practical range of the DP.

LPT stayed in the microsecond range in Figure 1, far below the DP at every size. At much larger sizes (Figure 3), its runtime grew roughly tenfold for each tenfold increase in n, from a few hundredths of a millisecond at n = 100 to around 50 ms at n = 100,000, consistent with its O(n log n) sort. The three lines are close together, so the range of job times had little effect compared with n, supporting our prediction that LPT's runtime depends on input size rather than the size of the job times. The one exception was at n = 100,000, where the 1–100 range was slightly faster.

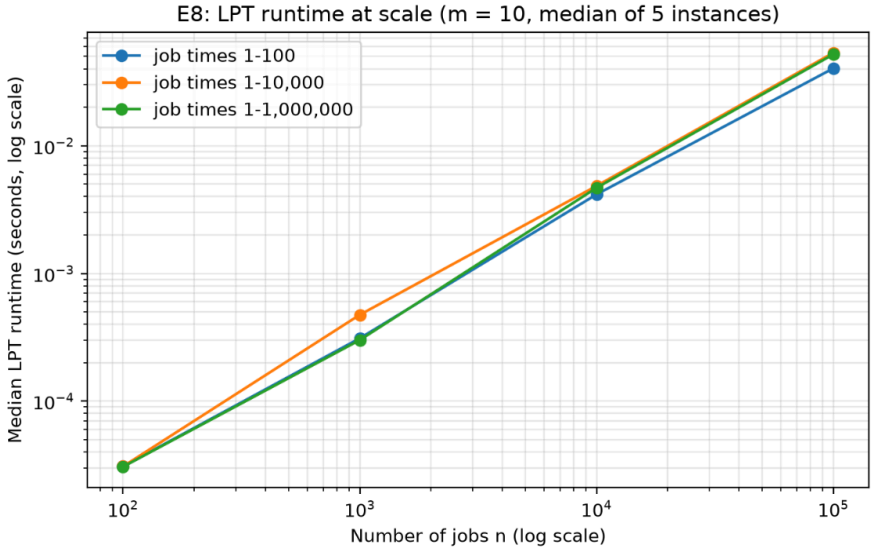


Figure 3: LPT's median runtime against n for the three ranges of job times (E8, m = 10)

Figure 4 shows that sorting machine loads produced the largest general improvement to the DP, reducing average runtime from about 32–33 ms for basic recursion or caching alone to 5.2 ms by merging symmetric states. Skipping equal-load machines provided only a small additional reduction because many equivalent states had already been removed. Early stopping reduced average runtime further to 3.6 ms, with the greatest benefit on balanced and repeated inputs where the lower bound could be reached quickly. Caching alone was sometimes slower than uncached recursion on uniform and balanced inputs due to cache overhead, but was beneficial on repeated inputs where states were reused more frequently.

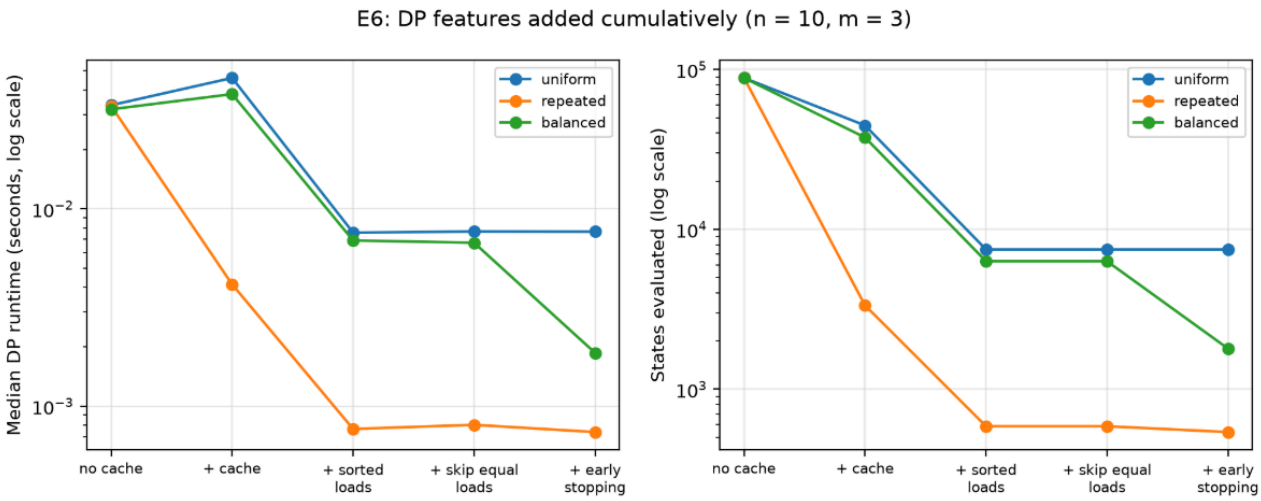


Figure 4: Effect of Cumulative DP Optimisations on Runtime and State Exploration

Taken together, these results explain the gap between the two algorithms. The DP's cost is set by how many distinct load states it has to store and revisit, and that number grows rapidly with each extra machine, because there are more ways to divide the same jobs. LPT never revisits a decision, sorting once and placing each job with a single heap operation, so its cost depends only on n and barely on m.

4.2 Solution Quality

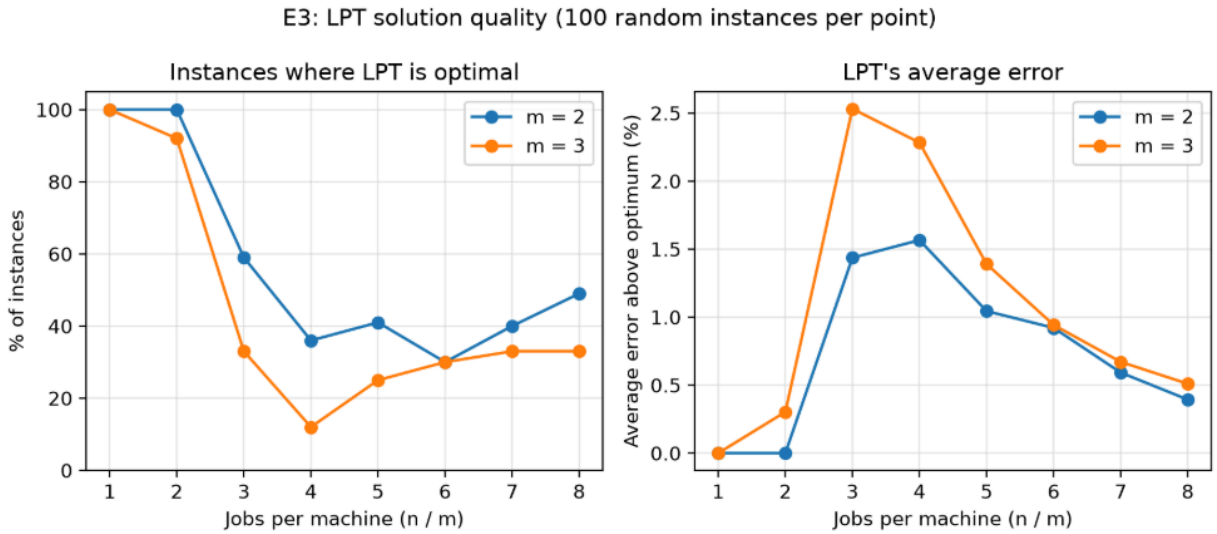


Figure 5: Percentage of instances where LPT is optimal (left) and LPT's average error (right), against jobs per machine (E3, 100 instances per point)

Figure 5 shows two different trends. With one or two jobs per machine, LPT was optimal on all or nearly all instances. From three jobs per machine onwards, it was optimal on well under half the instances for both machine counts, and this proportion didn't recover across the range we tested. Its average error peaked at

around three to four jobs per machine, at little over 2%, and then fell steadily towards 0.5%. LPT was usually slightly off, but only by a small amount. The largest single error across these instances was 9.43%, well below its worst case guarantee.

Both trends reflect the source of LPT's error, the final job assigned to the most heavily loaded machine. With only one or two jobs per machine, the longest first ordering tends to produce well-balanced assignments, leaving little room for error.With three or four jobs per machine, early assignments have a greater influence on the final loads, and poor pairings cannot be corrected later. As the number of jobs per machine increases, the final job becomes smaller relative to the total load, reducing the potential error. However LPT rarely balances the loads exactly, which explains why it remains suboptimal on most instances.

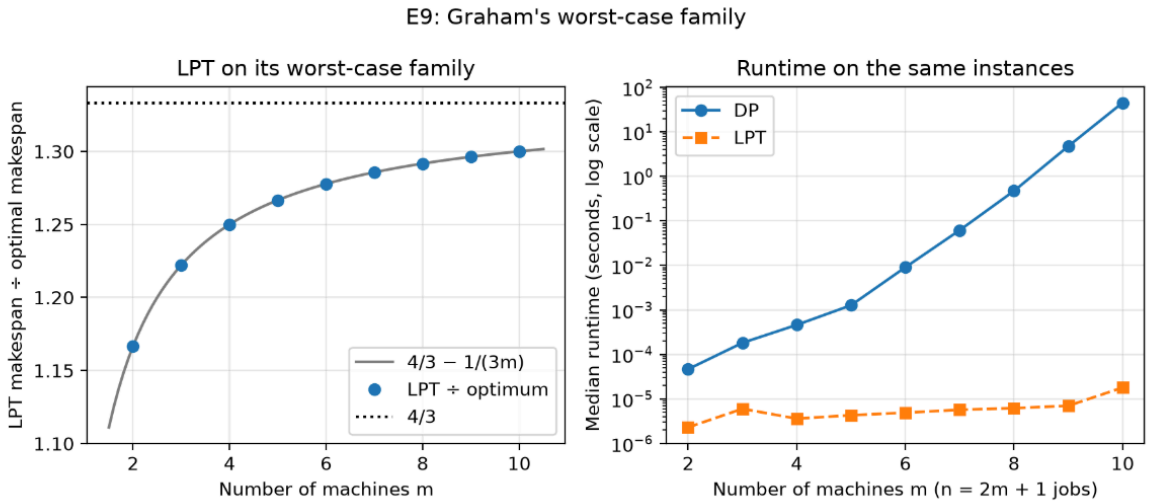


Figure 6: LPT's makespan divided by the optimum on Graham's worst case family, against the theoretical bound (left), and runtimes on the same instances (right).

On Graham's worst case family (Figure 6, left), every point lies exactly on the 4/3 - 1/(3m) curve, from 1.167 at m = 2 to 1.300 at m = 10. The worst case is therefore real, but it requires a specially constructed input. The random inputs in figure 5 never came close. The right panel shows the DP struggling on the same instances, rising steeply with each extra machine to about 45 seconds at m = 10, while LPT stays flat. These instances are designed so that the natural longest-first choices lead to the wrong schedule. The DP explores those same choices early, so it must search much of the space before finding the balanced schedule that lets it stop.

4.3 Effect of Input Structure

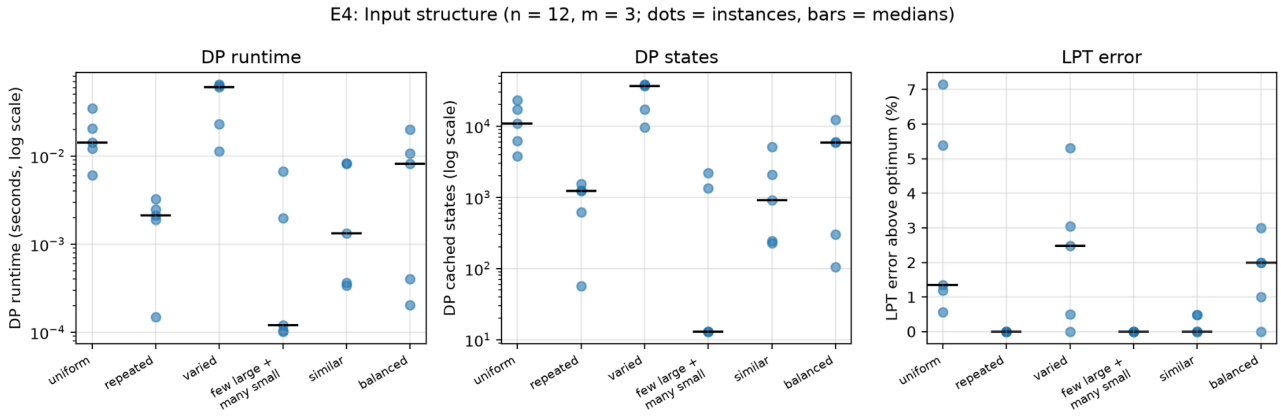


Figure 7: Effect on input structure of runtime and State Exploration.

E4 shows that input structure strongly affects the DP even with n = 12 and m = 3 fixed. Highly varied inputs were the most expensive, averaging about 27,969 states and 44.6 ms, while repeated-size and large-small instances averaged fewer than 1,000 states and around 2 ms. This indicates that repeated or structured job sizes create more reusable load states and reduce the search space. LPT stayed almost unchanged at roughly 4–5 µs across all families, while solution quality varied, with uniform inputs producing the largest average error at about 3.1%.

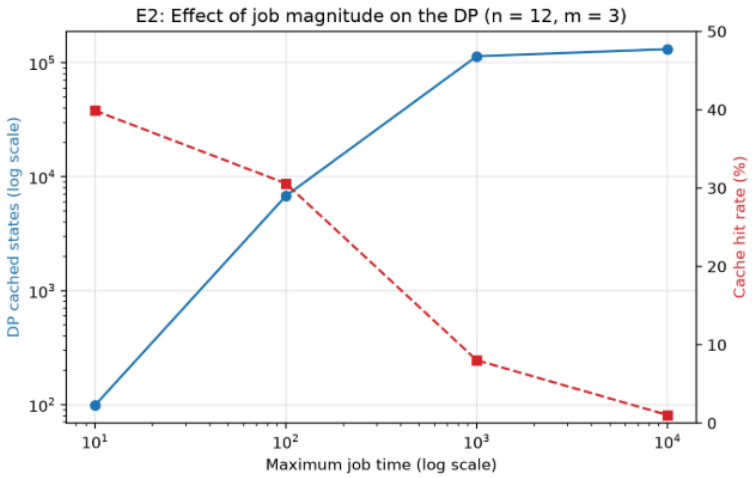


Figure 8: The DP's cached states and cache hit rate against the maximum job time.

Figure 8 shows how job size affects the DP. As the maximum job time increased, the number of states rose by around three orders of magnitude, while the cache hit rate fell towards zero. Larger, more varied times make it rare for two partial schedules to produce identical loads, so the cache stops helping. The state counts levels off at the largest job times, once almost every partial schedule already has unique loads. LPT is unaffected by job size because it only compares and adds job times, which costs the same regardless of size.

Figure 9 (E5) shows that lower-bound early stopping substantially improves the DP only when the bound is achievable. On balanced instances, average memoised states fell from about 15,752 to 3,620 and average median runtime dropped from 23.96ms to 4.73ms because the search could terminate once a schedule equal to the theoretical lower bound was found. In contrast, above-bound instances showed the same state and call counts with early stopping enabled or disabled, since the optimum exceeded the lower bound and the stopping condition was never reached. This supports the prediction that early stopping is highly effective when the lower bound is attainable, but provides little benefit otherwise.

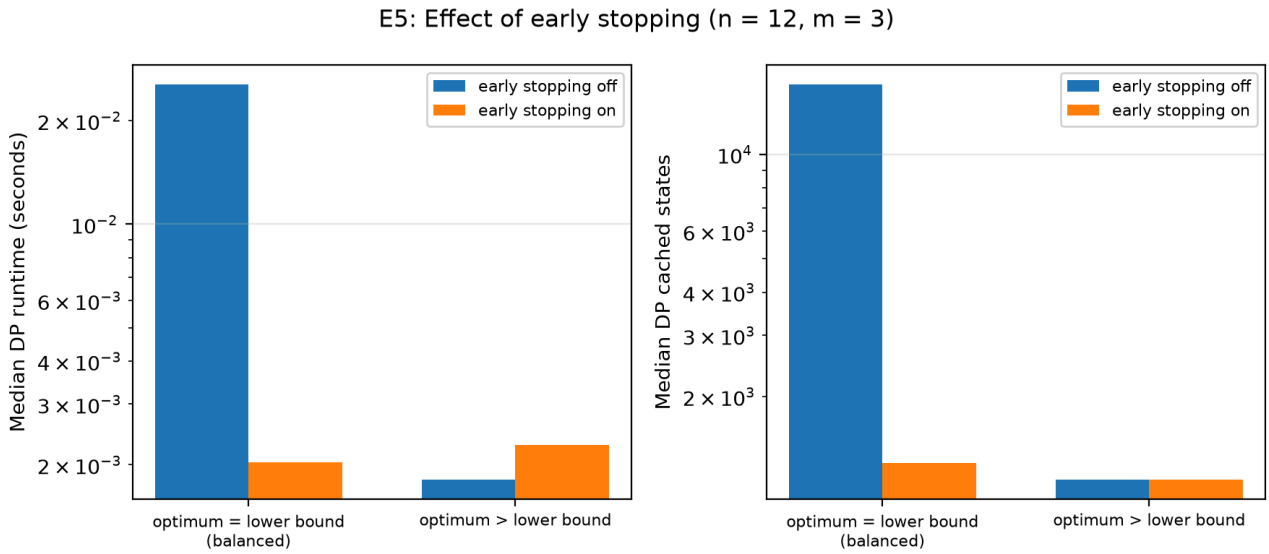


Figure 9: The effects of early stopping.

Figure 10 shows that job order can substantially affect DP performance while leaving the optimal makespan unchanged. Across all tested instances, descending order was fastest overall, averaging about 3.1 ms and 2,194 states, compared with 11.3 ms and 8,169 states for ascending order. The effect was strongest on uniform and balanced inputs, where processing larger jobs first allowed the DP to reach strong partial schedules and trigger pruning earlier. Repeated-size inputs were less sensitive to ordering because many states were already equivalent due to repeated processing times.

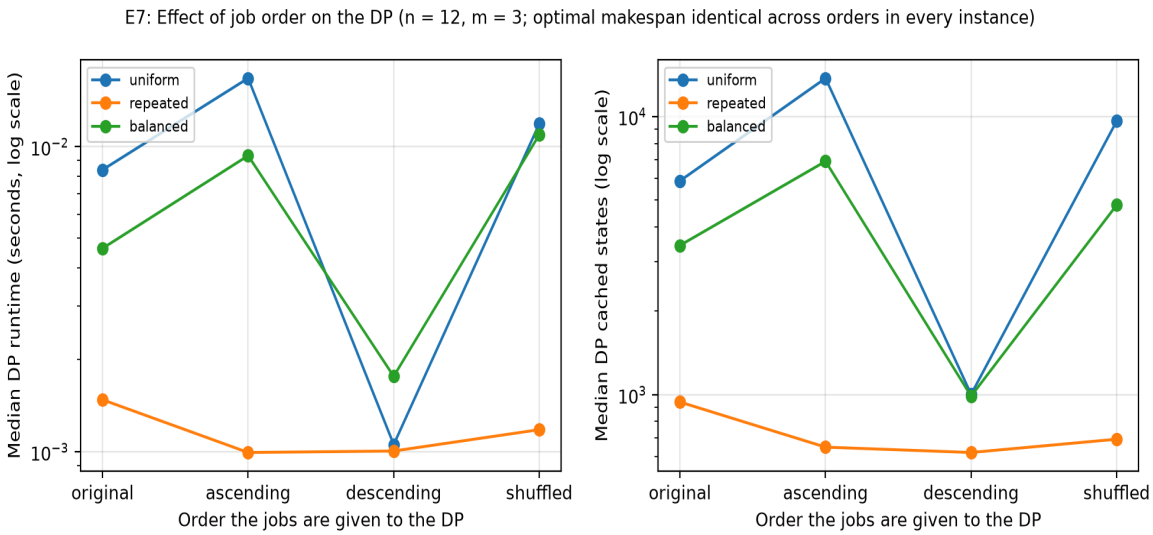


Figure 10: Effect of Job Ordering on DP Runtime

4.4 Which Algorithm Wins, and Where

LPT was faster than DP on every instance in every experiment, so the real comparison is between the DP's guaranteed optimum and its cost. The results divide into clear phases.

Phase	Better choice	Why	Evidence
Two or three machines up to ~40 jobs	DP	Few machines means few distinct load states, and early stopping usually ends the search quickly, so the exact answer costs milliseconds.	Figures 1, 2
Four or more machines, ~16+ jobs	LPT	The number of load states explodes with m, so the DP becomes unreliable within 10s.	Figures 1, 2
One or two jobs per machine	LPT	LPT is already optimal, so the DP's exactness adds nothing.	Figure 5
Three or four jobs per machine	DP	LPT's error is highest here, as early pairings can't be corrected.	Figure 5
Many jobs per machine	LPT for speed and DP for	LPT is usually within a fraction of percent, but rarely exact.	Figure 5

	exactness		
Large or highly varied job times	LPT	Few repeated loads means the DP's cache stops helping while LPT is unaffected.	Figures 7, 8
Repeated job sizes, or few large + many small	Either	Both do well, the DP reuses states and LPT is usually optimal.	Figure 7
Graham's family	Neither	LPT reaches its worst-case ratio, and the DP's search is led down the same misleading path.	Figure 6

In summary, the DP wins when the number of distinct load states stays small, which happens with few machines, small or repeated job times, or a balanced schedule that early stopping can find quickly. LPT wins everywhere else, and its typical error is small enough that the DP's extra cost is rarely justified beyond small instances.

5. What Surprised Us

5.1 DP

Adding more jobs sometimes made the DP faster, despite increasing the problem size.
Figure 1 showed that DP runtime did not increase consistently with the number of jobs. With three machines, runtime increased to several tens of milliseconds around n = 28 before decreasing for larger instances. A similar pattern appeared with four machines, although some instances exceeded the 10-second timeout.

Before running the experiments, we predicted that increasing the number of jobs would generally increase runtime and memory consumption. Each additional job introduces more possible assignments and potentially more distinct machine-load states, so we expected the computational cost to rise with input size.

The DP uses lower-bound early stopping, allowing it to terminate once a schedule matching the theoretical minimum is found. With more jobs, a perfectly balanced assignment can sometimes be easier to find, allowing the search to terminate before exploring most possible states. Since different job sets were generated at each size, this does not mean adding jobs to an existing instance necessarily improves performance. Instead, it demonstrates that DP runtime depends heavily on input structure and how quickly the lower bound is reached, rather than increasing consistently with job count.

A secondary unexpected finding was that memoisation alone sometimes made the DP slower. In Figure 4, caching increased average runtime from approximately 33 ms to 46 ms on uniform and balanced inputs, despite reducing repeated computation. This occurred because cache management introduced overhead that outweighed the savings when relatively few states could be reused. However, repeated-size inputs benefited substantially from memoisation, showing that its effectiveness depends on the amount of state reuse.

5.2 LPT

LPT was rarely exactly optimal, but almost always very close.

From three jobs per machine onwards, LPT was optimal on well under half of the instances, and this did not improve as jobs per machine increased (Figure 5, left). Yet over the same range, its average error fell steadily, to around 0.5%.

Before running the experiments, we predicted that LPT would match the optimum on most random instances, and that its error would be almost always zero at five or more jobs per machine. With more jobs per machine, the last job placed is small relative to each machine's load, so we expected LPT to balance the loads exactly in most cases.

With many jobs and whole-number times, a perfectly balanced schedule almost always exists, and the DP finds it exactly. LPT's last jobs are small, so they bring the loads very close to balance, but landing exactly on it requires the remaining jobs to fit the gaps precisely, which rarely happens. For example, with two machines and eight jobs per machine, the optimum is around 200, so an average error of about half a percent means LPT is typically off by just one unit. The two quantities measure different things: LPT's error shrinks because its last job becomes small relative to each machine's load, but rarely disappears.

6. What We Would Do Next

- Further pruning in the DP: Starting the DP with LPT's makespan as an upper bound would let it skip any branch that already exceeds it, and a tighter lower bound would stop rounding from blocking early stopping.
- Compare against other exact methods: Such as branch-and-bound or an ILP solver, to see whether the DP's limits come from the problem itself or from this particular approach.
- Larger and more adversarial benchmarks: We only tested the DP up to four machines on random inputs; since machine count dominated its cost, larger m would show how quickly its practical limit falls.
- More difficult LPT instances. Graham's family was hard for both algorithms, and longest-first order helped the DP on random inputs but not there. Finding other inputs that defeat LPT, and understanding why they also slow the DP, is an open question.

7. Individual Contributions

Thomas

- Primary author: exact DP
- DP implementation and testing

Anna

- Primary author: LPT approximation
- LPT implementation and testing

Shared

- Benchmark design.
- Input generation.
- Analysis.
- Report.
- Video

8. AI Use

8.1 Thomas

I used ChatGPT, Claude and Gemini throughout the assignment to assist with algorithm research, implementation, testing, and benchmarking.

ChatGPT was used to explore different exact approaches to makespan scheduling, including subset-based DP with bitmasking, and recursive DP. After comparing their complexity and implementation requirements, I ultimately went with a top-down approach with memoisation and sorted machine loads, which allowed equivalent states to be merged and support lower-bound early stopping. Ai also helped me understand these optimisations and their correctness.

For testing, I used adversarial prompts such as those suggested by Troy in the forum post regarding PA1 feedback. This helped me identify potential edge cases and weaknesses in my implementation.

Claude assisted with benchmarking scripts and suggested refactoring experiments E4 and E10 using shared infrastructure. I subsequently used Gemini to review Claude's refactor for potential issues, but it did not identify any significant problems. Further testing and manual inspection revealed inconsistencies in timeout handling and error classification, which I corrected. This demonstrated that AI-generated code and AI-based reviews cannot be assumed reliable without independent verification.

ChatGPT was used to generate an initial report template and seduction structure based on the requirements, it also critiqued my written drafts to help improve the clarity of my technical explanations. For the video presentation, ChatGPT suggested recommended time allocations for each section and critiqued draft slides for technical accuracy, and relevance to the marking criteria.

Overall AI was useful for exploring approaches, identifying potential issues, and helping with edge cases. However, I independently reviewed its suggestions, tested implementations and checked interpretations against our experimental results rather than accepting its outputs without verification.

8.2 Anna

I used Claude throughout the project. I started by using it to learn makespan scheduling and LPT, specifically the 4/3 guarantee and practice scenarios, which I solved by hand and checked. For my implementation, Claude gave me code skeletons which I filled out myself, including my first list-scan version of LPT and my test cases based on hand-worked examples. Claude then wrote the faster heap-based version of LPT, which I checked against those same tests. It also wrote the schedule validity checker, the input generators, the shared

timing helpers in common.py, the experiment scripts for E1, E2, E3, E8 and E9, and the plotting script. I ran all of the experiments on my laptop, and checked the code against our tests and the results it produced.

I also used Claude to plan the experiments from our predictions, to review Thomas's DP against our shared interface, and to draft parts of the report (Sections 2–6) and the video slides and script, which I then revised. When it drafted analysis from our results, I checked its numbers against the CSV files and figures, and asked it to rewrite the results section when it used values the figures did not show. Claude also helped me reword my writing at times, and the report was structured using Thomas's prompt, as mentioned in his section.

8.3 AI in Group Work

8.3.1 Where AI was wrong - Benchmarking Refactor

While working on the benchmarking experiments, Claude suggested refactoring E4 and E10 to use benchmarks/common.py, a shared benchmarking file developed by Anna. I accepted this suggestion and tested the refactored implementation, but found that it introduced inconsistencies in how experiments measured timeouts and handled errors. Unlike E5-E7, which waited for a worker readiness signal before starting the timeout, the refactored experiments started timing immediately after spawning the process. Additionally, worker crashes were incorrectly classified as timeouts, making the new results potentially misleading. To address these issues, I reviewed the existing benchmarking implementations and corrected the timeout handling to use a consistent starting point. I also separated genuine timeouts from worker errors to ensure failures were classified correctly. This experience demonstrated that AI-generated refactoring can introduce subtle errors even when the code appears functional, reinforcing the necessity of testing and reviewing suggested changes.

8.3.2 Where AI was wrong - Misleading Optimisation Advice

While developing my Dynamic Programming algorithm, I used ChatGPT to investigate optimisations such as memoisation, symmetry reduction and skipping machines with identical loads. ChatGPT suggested that these techniques would improve performance by reducing redundant computations. However, when I tested the optimisations individually in Experiment E6, I found that some actually increased execution time. For example, memoisation alone increased runtime from approximately 33 ms to 46 ms on uniform and balanced inputs, while skipping equal-load machines also introduced a small performance overhead in the tested cases. I identified this discrepancy by benchmarking different algorithm configurations and comparing their execution times. This demonstrated that AI can provide theoretically reasonable optimisation advice without adequately considering implementation overhead, reinforcing the importance of measuring performance rather than assuming an optimisation will be faster.

8.3.3 Where AI was wrong - Gemini Failing to Identify Errors in AI-Generated Code

I used Gemini to independently review Claude's benchmarking refactor for potential errors. Gemini identified no significant issues, but my subsequent testing and manual inspection revealed that inconsistent timeout handling and worker crashes were being misclassified as timeouts. I corrected these issues by standardising timeout measurements and separating worker errors from timeouts. This demonstrated that AI-based code reviews can overlook important bugs, reinforcing the need for independent verification.

8.4 Reflection of AI in group work

AI helped our group most with coordination. It helped us agree on a shared interface, write infrastructure we could both use (the input generators and timing helpers), and understand each other's algorithms well enough to write the comparison sections. It also made boilerplate, such as experiment scripts and plots, much faster to produce.

At the same time, it created risks specific to group work. Each of us used AI separately, so it could draft text about the other member's algorithm without knowing what they were currently working on, and its descriptions went out of date as their code changed (8.3.4). Shared code written by AI also spread its flaws into both members' experiments, and those flaws were only caught when one of us compared it against independently written code (8.3.1–8.3.2). An AI review of AI-written code did not catch them either, as Thomas found with Gemini. Finally, AI made it easy for one member to produce a large share of the infrastructure quickly, which made an honest account of who did what in Section 7 more important.

Overall, AI was most reliable when its output could be checked against something independent: our tests, a hand-worked example, or the other member's code.